uIME 产品手册

uIME Product Manual

uIME v0.1.0.51

2026-11-06

专有许可（二进制品个人用户免费）· Proprietary

UEFI Shell 中文输入法 · Chinese IME for the UEFI Shell

> **UEFI Shell 下的汉字输入法。** 以常驻驱动形态 `load` 进系统，接管控制台输入，把拼音翻译成汉字并以 Unicode 按键注入——**任何读控制台的 UEFI 程序都能收到汉字**，不需要为它改一行代码。

目录 · Contents

（若下方目录为空，请在 Word 中按 F9 或右键选择“更新域”）

（目录域——在 Word 中更新后显示）

中文 | Chinese

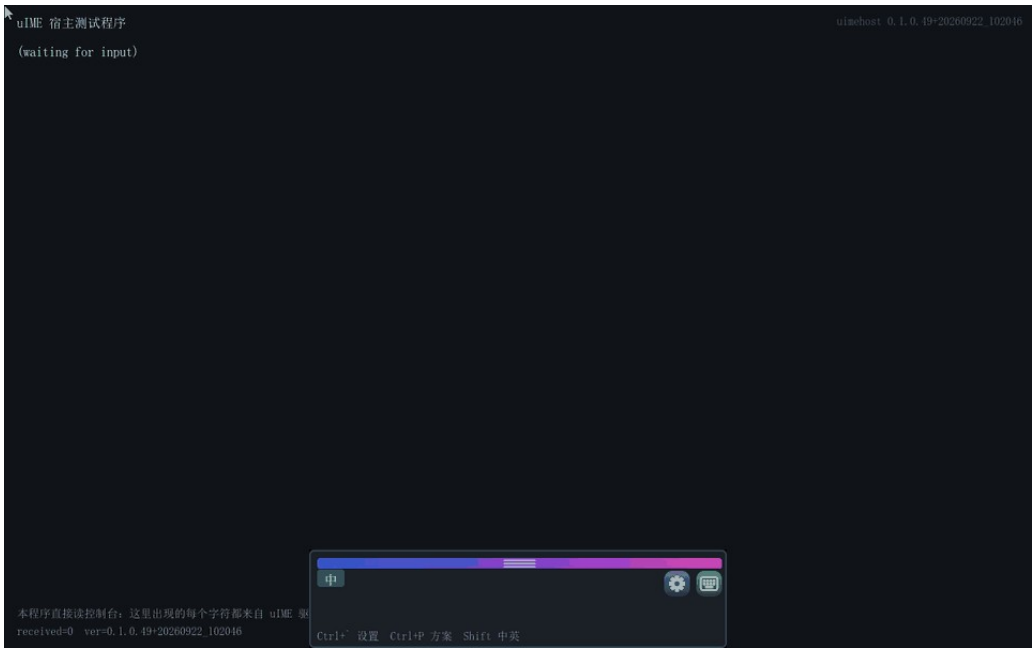
01 · 它解决什么

UEFI Shell 与绝大多数 UEFI 应用只认 ASCII：一次按键就是一个 `EFI_INPUT_KEY.UnicodeChar`，汉字（`U+4E2D` 这种）根本进不来。想在固件环境里编辑一个含中文的文件、在固件自带的编辑器里写中文，此前没有输入手段。

uIME 把「输入法」这整件事下沉到**控制台输入层**：逐个候选词的翻译发生在按键到达宿主**之前**，选中的汉字经注入队列变成一个 `UnicodeChar >= 0x80` 的按键——宿主读到它，与读到一个普通的 Unicode 按键毫无区别。



合成窗口浮在宿主程序之上。宿主照常打印自己的输出，uIME 的窗口盖在它上面——窗口**不是真窗口**，而是通过保存被覆盖的像素、在宿主每次重绘后重新盖章实现的。



完整演示（动图，真实 QEMU 录制）：宿主程序里打出「大小」「左右」并上屏 → 把窗口拖到左、右、屏幕中央、上方四个停靠位 → 打开设置逐项查看 → 用软键盘点出「南京」。动图在 Word 里只显示第一帧；要看完整动画，请打开仓库中的 `docs/manual/images/uime-demo.gif`，或在线手册 `docs/manual/index.html`。

02 · 快速上手

交付物只有 `uime.efi` **一个**。构建还会产出一个演示宿主，用来证明汉字真的到了**另一个**程序里；它不是产品的一部分，交付时可以不要。两者是两个角色，不是同一个程序的两个副本：

文件	类型	职责
`uime.efi` ~2.3 MB	**DXE_DRIVER**（常驻）	**输入法本体，也是唯一的交付物**。安装协议拦截层、建立悬浮窗、跑引擎。**它是常驻的**：入口返回后镜像不解体，包装留在系统里继续服务后续启动的程序。
`uimehost.efi` ~2.1 MB	UEFI 应用	**演示宿主**。一个最小的、会读控制台的程序，用来证明汉字真的到了**另一个**程序里。它不是产品的一部分，交付时可以不要。

怎么启动它？ 先 `fs0:` 建立当前目录再 `load uime.efi`；`startup.nsh` 里那一刻还没有当前目录，所以要写绝对路径 `load fs0:\uime.efi`。也可以把启动项指向 `uime.efi`（固件按 FilePath 加载），或把它编进固件卷——磁盘上什么都不用放。**不能**直接敲 `uime.efi`：Shell 的派发只接受应用镜像，驱动只能被 `load`、启动项或 dispatcher 启动。

启动顺序

顺序是**强制**的：驱动必须先于任何需要中文的应用程序。

```
Shell> fs0:                  :: 先建立当前目录（或直接写绝对路径）
FS0:\> load uime.efi         :: 装上并常驻（窗口随即在屏幕上）
Shell> uimehost.efi           :: 再启动需要中文输入的应用程序
```

- **1 · load uime.efi** — 驱动进城。串口出现 `[Uime] installed, resident` 即就绪。
- **2 · 出现窗口** — 屏幕底部中央浮出候选窗，左上角显示 **中**。
- **3 · 任意宿主程序** — 此后启动的程序只要读控制台，就能直接收到汉字。

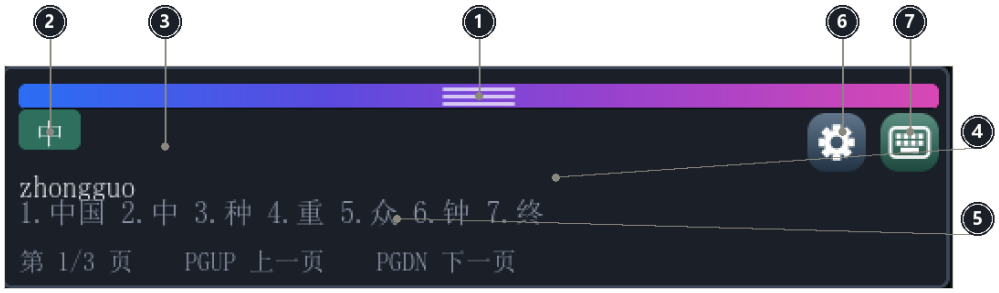
在自动启动脚本里：

```
:: startup.nsh
@echo -off
load fs0:\uime.efi          :: 装上并常驻（那一刻还没有当前目录，必须写绝对路径）
uimehost.efi                :: 演示宿主
```

卸载同样是**必须干净**的操作：失败时不报错，只是把控制台留在指向一块即将被释放的镜像的状态——那是**静默损坏**。用 dh 找到驱动的 image handle，再 `unload` 它的编号。

03 · 界面导览

候选窗是三段式：**状态片 + 拼音串 / 候选行 / 提示行**。下图标号与下面的表格一一对应。



候选窗各部件（在真实截图上标注）。窗口固定 520 像素宽；三种高度分别是 122（仅合成）、266（含软键盘）、334（设置对话框）。

部件	作用
① 拖动条	四色渐变（蔚蓝→靛→紫→洋红），中间三道半透明白握把纹。按住即可拖动窗口。它在左右两侧**避开**了两个角上的按钮——按钮顶部只比它低 2 像素，满宽会把点齿轮变成拖窗口。
② 中/英 状态片	显示当前中/英状态。它同时是个**点击目标**——点它切换中英，比记住 `Shift` / `Ctrl+Space` 好发现得多。
③ 拼音串	正在拼的内容。`Backspace` 退一个字母，`Esc` 丢弃整串。
④ 候选行	按词频排名的候选词。整串词优先，不足时用首音节单字补齐。数字键选字，也可以直接点。
⑤ 提示行	按当前状态说话的一行，见 **§05**。它的文字存在标签之外——窗口每次换高度都整个重建，只写在标签里的文字重建后是空的。
⑥ 齿轮（设置）	打开设置对话框，等价于 `Ctrl+`` ``。
⑦ 软键盘	展开/收起三行 QWERTY 软键盘。展开时窗口加高。

图标是位图，不是字形。 右上角两个按钮用的是距离场光栅化出来的 ARGB8888 位图，而不是字体里的符号——字库是 CJK 子集，键盘和齿轮这类符号不在覆盖范围内，用字形必然画成方框。

04 · 打字与选字



输入 zhongguo 后：拼音串在上，候选行给出「中国」，提示行讲当前能按什么键。

按键	作用
`a–z` `;`	拼音字母。`;` 是微软双拼方案里 `ing` 的位置。
`1–9`	选对应候选词并上屏。
`Space`	选第一个候选词。
`Enter`	放弃候选，把拼音原样上屏。
`Backspace`	退一个拼音字母；缓冲为空时透传给宿主。
`Esc`	丢弃整串拼音。
`PgDn` / `PgUp`	下一页 / 上一页候选。
`-` `=` `,` `.`	同样翻页（更早的键位，仍保留）。它们不再占提示行的位置。
`Ctrl+Space`	显示/隐藏窗口，并切换中英。
单击 `Shift`	切换中英。
`Ctrl+``	打开设置对话框。
`Ctrl+P`	循环切换输入方案。

英文模式下按键全部透传。 切到「英」之后 uIME 不吞任何键，宿主拿到的是原始按键——宿主自己的快捷键不会失灵。



中文模式下未在打字时的形态：拼音串与候选行都收走，状态片仍是「中」，提示行回到空闲档。

05 · 翻页找字

候选多于一页时，提示行会变成翻页提示——这正是「想找的字不在这页」这个问题的答案所在。



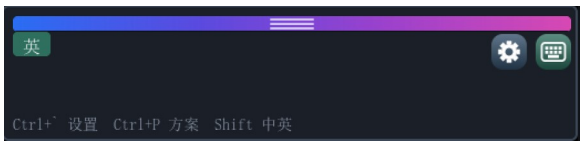
第 1/3 页：提示行显示页码与翻页键。



按 PgDn 后到第 2/3 页，候选行与提示行**同时**跟进。

提示行按状态切换，共三档：

状态	显示
没在打字	`Ctrl+` 设置　`Ctrl+P` 方案　`Shift` 中英
候选装得下一页	**数字键选字**　**空格** 选第一个　**回车** 上屏拼音
候选多页	`第 1/3 页`　`PGUP` 上一页　`PGDN` 下一页



空闲档：英文状态下的窗口，提示行给的是三个开关的快捷键。



单页档：`dia` 只有一个候选，提示行给的是选字键。

06 · 输入方案

四种方案用 `Ctrl+P` 循环，也可以在设置里直接选。

方案	打法	例
全拼	完整拼音逐个字母	`zhongguo` → 中国
双拼（小鹤 / 微软）	声母一键、韵母一键，两键一音节	`vsgo` → 中国
简拼	只打每个音节的声母	`zg` → 中国
五笔 86	按字根拆字，最多四码	`khlg` → 中国



小鹤双拼下用 `vsgo` 打出「中国」。



简拼下只用 `zg` 两个键。



五笔 86 下用 `khlg` 打出「中国」。字形码占着拼音行的位置，提示行也相应写成「上屏编码」。

双拼键位是从 RIME 公开方案机械推导的，不是手抄的。 **简拼按全局词频排名**。这一点不是实现细节：词表按首音节分桶、桶之间按音节字母序排列，如果按桶序取，zg 会先答出「糟糕」而不是「中国」。

五笔 86 是字形码，与拼音无关：一个字或二字词最多四码，一级简码一键出字，二级简码两键出字。编码有精确结果时，候选窗只列这几个候选；只有遇到没有对应简码的未完成码，才按词频给出以这几键开头的提示。敲满四码后再按字母，首选自动上屏，接着开始打下一个字。z 通配符与拼音反查不在这一版里。

07 · 软键盘

点右上角的**键盘图标**展开三行 QWERTY（27 键 = a–z 加 `;`）。这在只有鼠标的环境里是有用处的——UEFI Shell 的 USB 鼠标通常先于某种键盘可用。



展开后窗口从 520×122 加高到 520×266，期间右上角的软键盘按钮保持点亮状态。



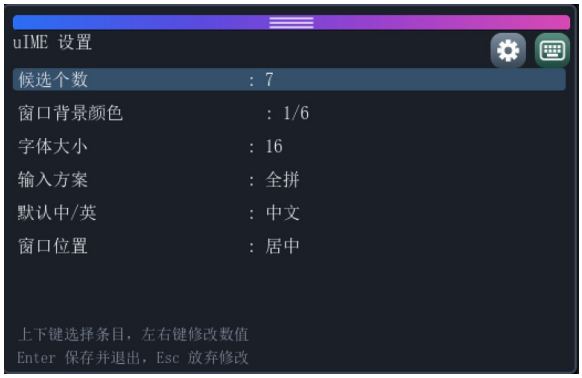
用鼠标点软键盘上的 z：拼音串立刻出现 z，提示行进到选字档——点键与物理按键走的是同一条路径。

点键与物理键盘走同一条引擎路径。 软键盘不是把字符直接塞给宿主，而是产生与按键等价的引擎输入——所以它同样有拼音串、候选、翻页，行为完全一致。

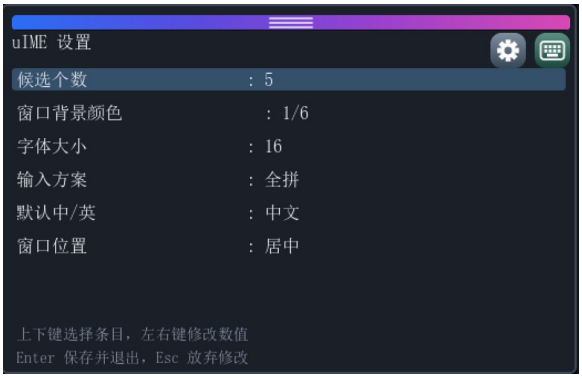
08 · 设置

点**齿轮图标**或按 `Ctrl+`` 打开。窗口底部有一行操作说明，不必记：

`↑↓` 选条目 · `←→` 改数值 · `Enter` 保存退出 · `Esc` 放弃



设置对话框：六项设置，窗口加高到 520×334。

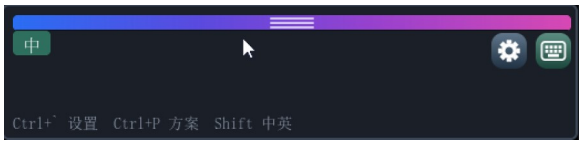


同一行「候选个数」在 ←→ 下由 7 变成 5——改数值是原地生效的。

设置项	取值	说明
候选个数	3 / 4 / **5** / 6 / 7 / 8	一页显示几个候选。
窗口背景	6 种	只改窗身配色；拖动条那道四色渐变是固定的 accent，不跟着变。
字体大小	14 / **16** / 18 / 20	候选行与提示行的字号。
输入方案	全拼 / 小鹤双拼 / 微软双拼 / 简拼 / 五笔	等价于 `Ctrl+P` 的选择。
默认输入模式	中 / 英	启动时的初始状态。
窗口位置	底部居中 / 左下 / 右下	之后的拖动会覆盖它。

09 · 拖动与停靠

按住拖动条把窗口拖到任意位置；**松手时离哪条边近就贴哪条**（32 像素以内）。



拖向左上后贴边停靠。



即使指针越过屏幕角落，窗口也**不会**甩到对面——抓取位移是有符号运算。

为什么拖动不会在屏幕上留一道。 窗口不是真窗口，"擦掉自己"只能靠盖上去之前存下来的那一份像素。位移**先画出新窗口、再归还旧矩形让出的部分**——屏幕上任意时刻都至少有一个完整窗口。若反过来先擦后画，两次之间会有一帧屏幕上根本没有窗口，而显示控制器是异步扫帧缓冲的，扫到那一帧就是一次闪烁。

10 · 浮在宿主之上

宿主程序完全不知道 uIME 的存在，而且往往每帧整屏重绘——重绘会把窗口盖掉。GOP 包装层在宿主的每次 `Blt` 之后把窗口重新盖章回去，并且**只重画被宿主改动的那块交集**：宿主刷一个光标，就只付光标那点代价。



宿主刚刚在自己的画面上画过东西（与 §01 同一场景）：窗口依旧完整地覆在它上面，因为包装层在每次 Blt 之后把窗口重新盖章回去。

泵点在宿主的按键调用上，没有定时器。 这带来一条硬约束：宿主如果**不轮询控制台**（例如卡在某种模态等待里），IME 窗口就不会动——这在 §13 已知限制里有完整说明。

11 · 配置文件

设置写入启动卷根目录的 `uime.cfg`，纯文本 `key=value`，可以直接用 Shell 的 `edit` 命令改。

```
candidates=5
background=0
font=16
scheme=0
chinese=1
placement=0
```

缺失或损坏的文件不是错误——启动时按默认值继续。卸载驱动时也会保存一次：等窗口已经擦掉再写文件是没有道理的，用户的改动不能因为卸载丢掉。

12 · 故障排查

现象	先看什么
窗口没出现	串口有没有 `[Uime] window shown`。没有的话看更早的行：`[UimeOverlay] install ok` 与 `[UimeOverlay] selftest ok` 必须都在——自检失败会回滚整个安装，而不是留一个坏掉的半安装状态。
窗口出来了但打不出候选	看提示行显示的状态：停在「英」的画并不意味着故障。`Ctrl+Space` 或点中/英状态片切回。
在 Shell 里敲命令被吃掉	中文模式下字母会进拼音缓冲——这是输入法在干活，不是故障。敲命令前先按 `Shift` 切英文。
候选里有方框	字库没覆盖这个字。词库已按字体 cmap 过滤到 GB2312 全量；GB2312 之外的生僻字确实打不出来。
宿主收不到汉字	确认宿主是通过**控制台**读键的。直接访问键盘控制器、绕过 `SimpleTextIn(Ex)` 的程序不走拦截层。
窗口冻住不动	宿主当前没在轮询控制台——那就是泵点。这条见 §13。
测试模式之间互相干扰	每个模式前重置磁盘镜像。`uime.cfg` 是会持久化的，上一轮的输入方案会被下一轮继承。

13 · 已知限制

- **宿主必须轮询控制台。** 没有定时器，宿主的 `ReadKeyStroke(Ex)` 调用既是输入口也是唯一的泵点。宿主卡在模态等待里时窗口冻住。
- **回读屏幕会读到窗口。** 宿主若用 `EfiBltVideoToBltBuffer` 读回，拿到的像素里包含盖在上面的窗口，因为栈里没有回读通路。
- **字库覆盖 GB2312 全量 6763 字**；GBK / Unicode 扩展区的生僻字仍打不出来。
- **中文模式下 Shell 自身的命令行也会被拦。** 按键先进输入法，所以装上之后敲 UEFI Shell 命令之前要先按 `Shift` 切英文——拦截式输入法的固有行为，不是缺陷。
- **本版明确不做**：模糊音（zh/z、n/l）、用户词频自学习、跟随光标定位。

uIME 不开源。 源码留在私有仓，不对外授权；对外发布的 `uime.efi` 二进制品**免费授权个人用户**用于个人、非商业用途，商业用途需另行取得授权（条款见 `LICENSE`）。词库数据来自 `pinyin-data` / `phrase-`

`pinyin-data` / `jieba`（均为 MIT，版权归各自作者）；五笔 86 码表来自 `limaofu/cof86`（MIT）。Rime 的 LGPL 词库被明确排除以避免许可污染。集成方式见随附的《uIME 集成指南》（Word / PDF）。

English | 英文

English

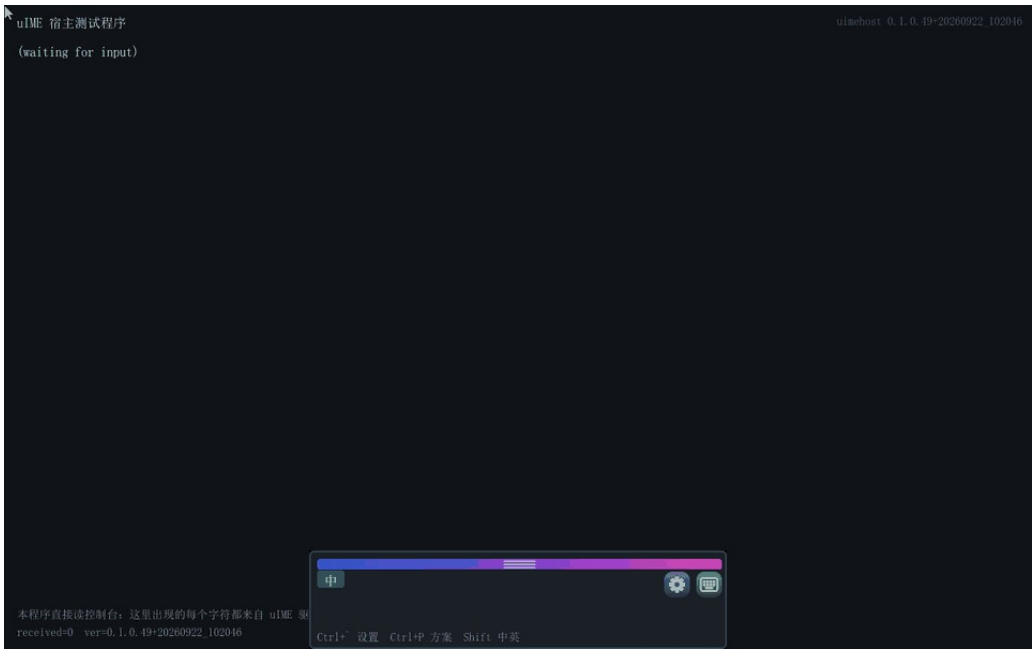
01 · The problem

UEFI consoles speak ASCII. A keystroke is an `EFI_INPUT_KEY.UnicodeChar`, and a Han character like `U+4E2D` never arrives — which left no way to author Chinese content inside firmware.

uIME pushes the input method down to the **console-input layer**. Translation happens **before** the key reaches the host, and committed characters leave through an injection queue as `UnicodeChar >= 0x80` keys — indistinguishable, to the host, from any other keystroke.



The composition window floats above the host. The host prints as usual; uIME draws over it. The window is **not a real window** — it exists because the overlay saves the pixels it covers and re-stamps itself after every host repaint.



Full demo (animated GIF, recorded from a real QEMU run): typing 「大小」 and 「左右」 into the host and committing them → dragging the window to the left, right, centre and top docks → opening settings and walking the options → tapping 「南京」 out on the soft keyboard. Word shows only the first frame of a GIF; for the animation open `docs/manual/images/uime-demo.gif` in the repository, or the online manual `docs/manual/index.html`.

02 · Getting started

The deliverable is **one** file: `uime.efi`. The build also produces a demonstration host, used to prove the characters really reach **another** program; it is not part of the product and can be left out of a shipment. Two roles, not two copies of one program:

Image	Module type	Role
`uime.efi` ~2.3 MB	**DXE_DRIVER**, resident	**The input method - and the only deliverable**. Installs the interception layer, creates the floating window, runs the engine. **It stays resident** — returning from the entry point does not tear it down.
`uimehost.efi` ~2.1 MB	UEFI application	**Demonstration host**. A minimal console-reading program that proves the characters really reach **another** application. Not part of the product — ship without it.

How do you start it? Run `fs0:` first, then `load uime.efi`. Inside `startup.nsh` there is no current directory yet, so use an absolute path: `load fs0:\uime.efi`. A boot option pointing at `uime.efi` works too (the firmware loads it by FilePath), and so does compiling it into a firmware volume - nothing on disk at all. **Typing** `uime.efi` does **not** work: the Shell only dispatches application images; a driver is started by `load`, a boot option, or the dispatcher.

Boot order

The order is **mandatory**: the driver must precede any application that wants Chinese.

```
Shell> fs0:                 :: install a cwd first (or spell the path absolutely)
FS0:\> load uime.efi        :: install, resident (window appears)
Shell> uimehost.efi          :: then start whatever application needs Chinese
```

- **1 · load uime.efi** — The driver lands. Ready when the log shows `[Uime] installed, resident`.
- **2 · the window** — The composition window appears bottom-centre, showing 中.
- **3 · any host** — Any application started afterwards receives Chinese simply by reading the console.

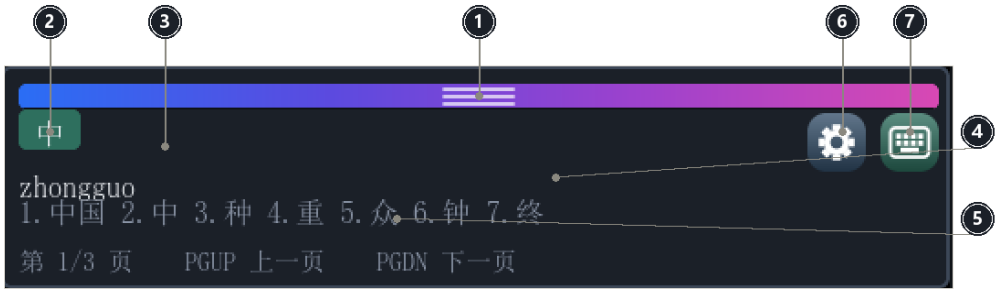
In the automated boot script:

```
:: startup.nsh
@echo -off
load fs0:\uime.efi          :: install, resident (no cwd yet, so spell the path absolutely)
uimehost.efi                :: demonstration host
```

Unloading has to be clean too: its failure is not an error but **silent corruption** — the console left pointing into an image that is about to be discarded. Find the driver's image handle with `dh`, then `unload` its number.

03 · The window

The window is a three-part stack: **status chip + pinyin line / candidate row / hint row**. The numbers in the figure match the table below.



Parts of the composition window, annotated on a real capture. It is a fixed 520px wide; the three heights are 122 (composition only), 266 (with the soft keyboard) and 334 (settings).

Part	What it does
① Grab bar	Four-colour gradient (azure → indigo → purple → magenta) with three translucent white grip lines. Drag the window by it. It deliberately **stops short** of the two corner buttons — their tops sit only 2px below, and a full-width bar would turn a click on the gear into a drag.
② Language chip	Shows the current Chinese/English mode, and is also a **click target** that toggles it — far more discoverable than remembering `Shift` or `Ctrl+Space`.
③ Pinyin line	What is being spelled. `Backspace` removes one letter, `Esc` discards the buffer.
④ Candidate row	Candidates ranked by word frequency — whole words first, then single characters matching the leading syllable. Select with number keys or by clicking.
⑤ Hint row	A line that reports the keys that matter right now; see **§05**. Its text lives outside the label, because the window is rebuilt wholesale on every height change.
⑥ Gear (settings)	Opens the settings dialog, same as `Ctrl+`` ``.
⑦ Soft keyboard	Shows or hides a three-row QWERTY soft keyboard, growing the window when shown.

The icons are bitmaps, not glyphs. The two corner buttons use ARGB8888 bitmaps rasterised from a signed distance field rather than font glyphs: the fonts are CJK subsets with no keyboard or gear in them, and a missing glyph draws as an empty box.

04 · Typing & selecting



After typing `zhongguo`: the pinyin line on top, 「中国」in the candidate row, and the hint row listing the keys that do something.

Key	Effect
`a–z` `;`	Pinyin letters. `;` is where Microsoft double-pinyin puts `ing`.
`1–9`	Commit the numbered candidate.
`Space`	Commit the first candidate.
`Enter`	Dismiss candidates and commit the raw **pinyin**.
`Backspace`	Delete one pinyin letter; passed to the host when the buffer is empty.
`Esc`	Discard the composition.
`PgDn` / `PgUp`	Next / previous candidate page.
`-` `=` `,` `.`	The same paging, from the earlier binding. Still supported, no longer advertised in the hint row.
`Ctrl+Space`	Show/hide the window and toggle Chinese/English.
Single `Shift`	Toggle Chinese/English.
`Ctrl+``	Open the settings dialog.
`Ctrl+P`	Cycle the input scheme.

English mode passes every key through. Switched to English, uIME swallows nothing and the host sees raw keystrokes — its own shortcuts keep working.



The idle shape in Chinese mode: composition and candidate rows are both withdrawn, the chip still reads 「中」, and the hint row falls back to the hotkey state.

05 · Paging

When there is more than one page, the hint row turns into a paging prompt — which is exactly the answer to “the character I want isn’t on this page”.



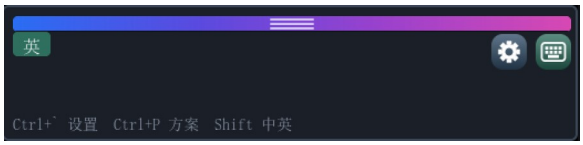
Page 1/3: the hint row shows the page number and the paging keys.



After `PgDn`, page 2/3 — candidates and hint row follow **together**.

The hint row has three states:

State	Shows
Idle	`Ctrl+` settings `Ctrl+P` scheme `Shift` 中/英
One page	number key to select `Space` first `Enter` commits pinyin
Several pages	page 1/3 `PGUP` previous `PGDN` next



Idle state, English mode: the hint row gives the three toggles.

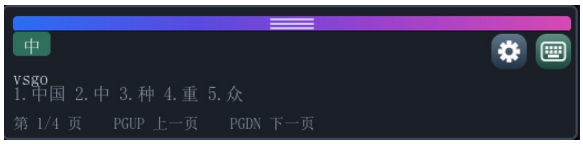


One page: `dia` has a single candidate, so the hint gives the selection keys.

06 · Input schemes

Cycle the four schemes with `Ctrl+P`, or pick one in the settings.

Scheme	How you type	Example
Full pinyin	Spell the whole syllable	`zhongguo` → 中国
Double pinyin (Xiaohe / Microsoft)	One key for the initial, one for the final	`vsgo` → 中国
Jianpin	Initials only, one key per syllable	`zg` → 中国
Wubi 86	Shape code, up to four keys	`khlg` → 中国



Xiaohe double-pinyin: `vsgo` yields 「中国」.



Jianpin: just `zg`.



Wubi 86: `khlg` yields 「中国」. The shape code takes the pinyin line, and the hint row names the raw code instead.

The double-pinyin tables are derived mechanically from RIME's public schemas, not transcribed by hand. **Jianpin ranks by global word frequency.** That is not a detail: the dictionary is bucketed by leading syllable with the buckets in alphabetical order, so taking them in bucket order answers 「糟糕」 before 「中国」.

Wubi 86 encodes a character's shape, not its reading. A character or a two-character word takes at most four keys; level-1 jianma produce a character from a single key and level-2 from two. When a code has an exact answer, the window lists only those candidates; longer codes are suggested by frequency only for an unfinished code with no exact match. Once four keys are down, the next letter commits the first candidate and starts the next code, so words flow without a space. The `z` wildcard and pinyin reverse lookup are not in this build.

07 · Soft keyboard

Press the **keyboard icon** to unfold three rows of QWERTY (27 keys: `a`–`z` plus `;`). This matters where only a pointer is available — a USB mouse often works in UEFI before any given keyboard does.



The window grows from 520×122 to 520×266, with the soft-keyboard button lit while it is open.



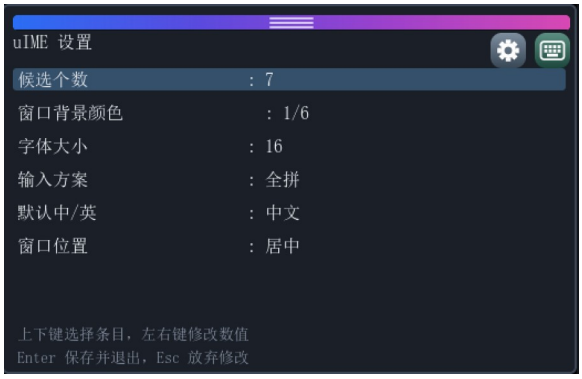
Clicking `z` on the soft keyboard: the pinyin line immediately shows `z` and the hint row moves to the selection state — a clicked key takes the same path as a physical one.

Key presses go through the same engine path as physical keys. The soft keyboard does not hand characters to the host directly; it produces engine input equivalent to a keystroke — so it has the same composition line, candidates and paging, and behaves identically.

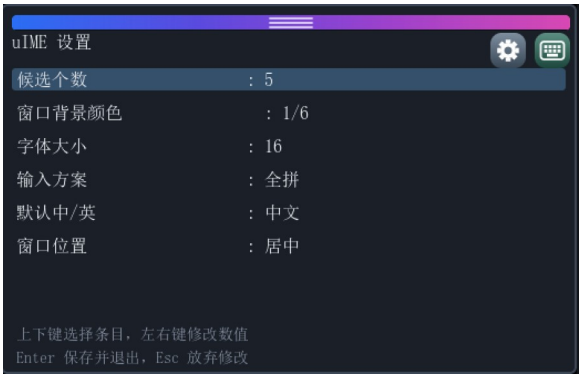
08 · Settings

Open with the **gear icon** or `Ctrl`+`` ` ``. There is a reminder printed along the bottom, so nothing has to be memorised:

`↑` `↓` select a row · `←` `→` change the value · `Enter` save and exit · `Esc` discard



Six rows; the window grows to 520×334.

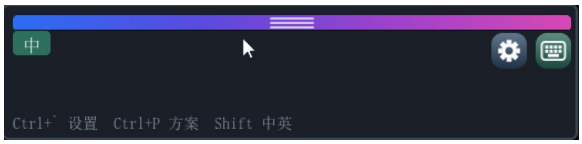


The same row, `←` `→`: 「候选个数」 goes from 7 to 5 in place.

Setting	Values	Notes
Candidate count	3 / 4 / **5** / 6 / 7 / 8	How many candidates fit a page.
Background	6 presets	Changes the panel only; the four-colour grab bar is a fixed accent and stays put.
Font size	14 / **16** / 18 / 20	Size of the candidate and hint rows.
Scheme	Full / Xiaohe / Microsoft / Jianpin / Wubi 86	The same choice `Ctrl+P` cycles.
Default mode	Chinese / English	Initial state at load time.
Placement	Bottom-centre / bottom-left / bottom-right	Dragging afterwards overrides it.

09 · Moving & docking

Drag the window by its bar; **release near an edge (within 32px) and it snaps flush** to that edge.



Docked to the top-left corner.



Even dragged past the corner it does **not** fly to the opposite side — the grab offset is computed signed.

Why dragging leaves no smear. There is no real window here, so erasing itself is only possible by restoring the pixels it saved before covering them. Each move **draws the new position first, then returns the strip the old rectangle vacates** — so a whole window is on screen at every instant. Doing it the other way round leaves a gap during which no window exists at all, and since the display controller scans asynchronously, hitting that gap is a visible flicker.

10 · Above the host

The host knows nothing about uIME and commonly repaints the whole screen each frame, which would paint the window out. The GOP wrapper re-stamps it after every host `Blt` — and **only over the intersection the host actually changed**: repainting a cursor costs a cursor-sized re-stamp.



The host has just painted over its own area (same scene as §01): the window is still whole and on top, because the wrapper re-stamps it after every blit.

The pump is the host's key-polling call; there is no timer. That imposes one hard constraint: a host that does **not** poll the console leaves the IME window frozen. See §13 for the full picture.

11 · uime.cfg

Settings are written to `uime.cfg` in the root of the boot volume — plain `key=value` text you can edit with the Shell's `edit`.

```
candidates=5
background=0
font=16
scheme=0
chinese=1
placement=0
```

A missing or corrupt file is not an error — startup falls back to defaults. Settings are also saved on unload: losing the user's changes would make the dialog pointless.

12 · Troubleshooting

Symptom	Check
No window	Does the log contain `[Uime] window shown`? If not, read back further: `[UimeOverlay] install ok` and `[UimeOverlay] selftest ok` must both be present — a failed self-test rolls the whole install back rather than leaving a half-installed one.
Window shows, no candidates	Read the hint row: sitting in English mode is not a fault. Toggle back with `Ctrl+Space` or the language chip.
Shell commands get eaten	In Chinese mode the letters go into the pinyin buffer — the input method working, not a fault. Press `Shift` for English before typing a command.
Empty boxes in candidates	The font lacks the glyph. The dictionary is filtered against the font cmap up to full GB2312; anything outside it genuinely cannot be typed.
Host receives nothing	Confirm the host reads through the **console**. Programs that talk to the keyboard controller directly, bypassing `SimpleTextIn(Ex)`, miss interception entirely.
Window frozen	The host is not polling the console, which *is* the pump. See §13.
Modes interfere	Reset the disk image before each mode. `uime.cfg` persists, so one run's scheme is inherited by the next.

13 · Known limits

- **The host must poll the console.** There is no timer; the host's `ReadKeyStroke(Ex)` call is both the input path and the only pump. A host blocked in a modal wait leaves the window frozen.
- **Reading the screen back includes the window.** A host using `EfiBltVideoToBltBuffer` gets pixels with the window composited in; the stack has no read-back path.
- **Font coverage is full GB2312** (6763 characters); rarer characters from GBK and beyond still cannot be typed.
- **In Chinese mode the Shell's own command line is intercepted too.** Keys reach the input method first, so press `Shift` for English before typing a UEFI Shell command — inherent to an intercepting input method, not a defect.

- **Deliberately out of scope for v1**: fuzzy initials (zh/z, n/l), user-dictionary frequency learning, and cursor-following placement.

uIME is not open source. Its source stays private and is not licensed to anyone; the published `uime.efi` binary is **free for individual users** for personal, non-commercial use, while commercial use needs a separate licence (see `LICENSE`). Dictionary data from `pinyin-data` / `phrase-pinyin-data` / `jieba` (all MIT, © their authors); the Wubi 86 code table comes from `limaofu/cof86` (MIT). Rime's LGPL dictionaries are deliberately excluded. Integration details live in the bundled uIME Integration Guide (Word / PDF).